

1. Understand the Architecture

The basic flow is:

Browser



Next.js Page / Component



API Route



MongoDB Driver



MongoDB Database

For example, when you submit a registration form:

User fills form



POST /api/users



Next.js API Route



MongoDB insertOne()



MongoDB



Response



Frontend

2. Create Next.js Project

If you don't already have a project:

```
npx create-next-app@latest my-next-app
```

Choose JavaScript when prompted.

Then:

```
cd my-next-app
```

```
npm run dev
```

Your application will run at:

http://localhost:3000

3. Install MongoDB Package

Install the official MongoDB Node.js driver:

```
npm install mongodb
```

Your package.json will contain something similar to:

```
{  
  "dependencies": {  
    "mongodb": "^..."  
  }  
}
```

4. Start MongoDB

If MongoDB is installed locally, make sure the MongoDB server is running.

Your local MongoDB connection will generally be:

```
mongodb://127.0.0.1:27017
```

For example, MongoDB has a database called:

```
myapp
```

and inside it we'll create:

```
users
```

```
collection.
```

The structure will be:

```
MongoDB
```

```
└─ myapp
```

```
  └─ users
```

```
    └─ document 1
```

```
    └─ document 2
```

```
    └─ document 3
```

5. Create Environment Variables

Create:

```
.env.local
```

at the root of your Next.js project.

```
MONGODB_URI=mongodb://127.0.0.1:27017
```

MONGODB_DB=myapp

Your project:

my-next-app/

```
|
|— app/
|— public/
|— .env.local
|— package.json
└─ ...
```

Important

Don't use:

NEXT_PUBLIC_MONGODB_URI=...

Your MongoDB connection string should **not** be exposed to the browser.

Use:

MONGODB_URI=...

6. Create MongoDB Connection File

Create:

lib/mongodb.js

So your structure becomes:

```
my-next-app/
|
|— app/
|
|— lib/
|   └─ mongodb.js
|
|— .env.local
└─ package.json
```

Put this inside mongodb.js:

```
import { MongoClient } from "mongodb";
```

```
const uri = process.env.MONGODB_URI;
```

```
if (!uri) {  
  throw new Error("MONGODB_URI is not defined");  
}  
  
let client;  
let clientPromise;  
  
if (process.env.NODE_ENV === "development") {  
  if (!global._mongoClientPromise) {  
    client = new MongoClient(uri);  
    global._mongoClientPromise = client.connect();  
  }  
  
  clientPromise = global._mongoClientPromise;  
} else {  
  client = new MongoClient(uri);  
  clientPromise = client.connect();  
}  
  
export default clientPromise;
```

7. Understand This Connection Code

This line:

```
const uri = process.env.MONGODB_URI;
```

gets:

```
MONGODB_URI=mongodb://127.0.0.1:27017
```

Then:

```
client = new MongoClient(uri);
```

creates a MongoDB client.

And:

```
client.connect();
```

connects Next.js to MongoDB.

We export:

```
export default clientPromise;
```

so other server-side files can use the connection.

8. Create API Route

Now create:

app/api/users/route.js

Full structure:

```
app/  
└─ api/  
    └─ users/  
        └─ route.js
```

Put:

```
import clientPromise from "@lib/mongodb";  
  
export async function GET() {  
  try {  
    const client = await clientPromise;  
  
    const db = client.db(process.env.MONGODB_DB);  
  
    const users = await db  
      .collection("users")  
      .find({})  
      .toArray();  
  
    return Response.json(users);  
  } catch (error) {  
    console.error(error);  
  
    return Response.json(  
      {  
        error: "Failed to fetch users",  
      },  
      {  
        status: 500,  
      }  
    );  
  }  
}
```

```
}  
);  
}  
}
```

Now your API endpoint is:

GET <http://localhost:3000/api/users>

9. What Happens When You Call /api/users?

The flow is:

Browser

|

| GET /api/users

↓

Next.js

|

↓

app/api/users/route.js

|

↓

clientPromise

|

↓

MongoDB Client

|

↓

myapp database

|

↓

users collection

|

↓

find({})

|

↓

Documents

|

↓

Response.json(users)

|

↓

Browser

10. Test the GET API

First run:

```
npm run dev
```

Then open:

```
http://localhost:3000/api/users
```

If there are no users, you might get:

```
[]
```

That's correct.

11. Create POST API

Now we want to insert a user.

Update:

```
app/api/users/route.js
```

to:

```
import clientPromise from "@/lib/mongodb";
```

```
export async function GET() {
```

```
  try {
```

```
    const client = await clientPromise;
```

```
    const db = client.db(process.env.MONGODB_DB);
```

```
    const users = await db
```

```
      .collection("users")
```

```
      .find({})
```

```
      .toArray();
```

```
    return Response.json(users);
  } catch (error) {
    console.error(error);

    return Response.json(
      { error: "Failed to fetch users" },
      { status: 500 }
    );
  }
}

export async function POST(request) {
  try {
    const body = await request.json();

    const client = await clientPromise;

    const db = client.db(process.env.MONGODB_DB);

    const user = {
      name: body.name,
      email: body.email,
      age: body.age,
    };

    const result = await db
      .collection("users")
      .insertOne(user);

    return Response.json(
      {
        message: "User created successfully",
        userId: result.insertedId,
      }
    );
  } catch (error) {
    console.error(error);
    return Response.json(
      { error: "Failed to create user" },
      { status: 500 }
    );
  }
}
```

```
    },  
    {  
      status: 201,  
    }  
  );  
} catch (error) {  
  console.error(error);  
  
  return Response.json(  
    { error: "Failed to create user" },  
    { status: 500 }  
  );  
}  
}
```

12. Test POST

You can use Postman, Thunder Client, or frontend JavaScript.

Request:

POST /api/users

Body:

```
{  
  "name": "John",  
  "email": "john@gmail.com",  
  "age": 25  
}
```

MongoDB will store something like:

```
{  
  "_id": "ObjectId(...)",  
  "name": "John",  
  "email": "john@gmail.com",  
  "age": 25  
}
```

13. Connect Frontend to API

Now create a frontend.

For example:

app/page.js

"use client";

import { useState } from "react";

export default function Home() {

const [name, setName] = useState("");

const [email, setEmail] = useState("");

const [age, setAge] = useState("");

async function createUser(e) {

e.preventDefault();

const response = await fetch("/api/users", {

method: "POST",

headers: {

"Content-Type": "application/json",

},

body: JSON.stringify({

name,

email,

age: Number(age),

}},

});

const data = await response.json();

console.log(data);

}

return (

<main>

```
<h1>Create User</h1>
```

```
<form onSubmit={createUser}>
```

```
  <input
```

```
    type="text"
```

```
    placeholder="Name"
```

```
    value={name}
```

```
    onChange={(e) => setName(e.target.value)}
```

```
  <input
```

```
    type="email"
```

```
    placeholder="Email"
```

```
    value={email}
```

```
    onChange={(e) => setEmail(e.target.value)}
```

```
  <input
```

```
    type="number"
```

```
    placeholder="Age"
```

```
    value={age}
```

```
    onChange={(e) => setAge(e.target.value)}
```

```
  <button type="submit">
```

```
    Create User
```

```
  </button>
```

```
</form>
```

```
</main>
```

```
);
```

```
}
```

Now the complete flow is:

User



Fills Name / Email / Age



Clicks Create User



createUser()



fetch("/api/users")



POST request



route.js



MongoDB



insertOne()



MongoDB saves document



Response



Frontend

14. GET Data in Frontend

You can also fetch users:

```
"use client";
```

```
import { useEffect, useState } from "react";
```

```
export default function Home() {
```

```
  const [users, setUsers] = useState([]);
```

```
  useEffect(() => {
```

```
    getUsers();
```

```
  }, []);
```

```

async function getUsers() {
  const response = await fetch("/api/users");

  const data = await response.json();

  setUsers(data);
}

return (
  <main>
    <h1>Users</h1>

    {users.map((user) => (
      <div key={user._id}>
        <h3>{user.name}</h3>
        <p>{user.email}</p>
        <p>{user.age}</p>
      </div>
    ))}
  </main>
);
}

```

15. Add UPDATE API

You can create:

PUT /api/users/:id

For example:

app/api/users/[id]/route.js

```
import { ObjectId } from "mongodb";
```

```
import clientPromise from "@/lib/mongodb";
```

```
export async function PUT(request, { params }) {
```

```
  try {
```

```
const { id } = await params;
```

```
const body = await request.json();
```

```
const client = await clientPromise;
```

```
const db = client.db(process.env.MONGODB_DB);
```

```
const result = await db.collection("users").updateOne(
  {
    _id: new ObjectId(id),
  },
  {
    $set: {
      name: body.name,
      email: body.email,
      age: body.age,
    },
  }
);
```

```
return Response.json({
  message: "User updated",
  modifiedCount: result.modifiedCount,
});
```

```
} catch (error) {
  console.error(error);
```

```
return Response.json(
  {
    error: "Failed to update user",
  },
  {
    status: 500,
```

```
}  
);  
}  
}
```

16. Add DELETE API

Same file:

app/api/users/[id]/route.js

```
import { ObjectId } from "mongodb";
```

```
import clientPromise from "@/lib/mongodb";
```

```
export async function DELETE(request, { params }) {  
  try {  
    const { id } = await params;  
  
    const client = await clientPromise;  
  
    const db = client.db(process.env.MONGODB_DB);  
  
    const result = await db.collection("users").deleteOne({  
      _id: new ObjectId(id),  
    });  
  
    return Response.json({  
      message: "User deleted",  
      deletedCount: result.deletedCount,  
    });  
  } catch (error) {  
    console.error(error);  
  
    return Response.json(  
      {  
        error: "Failed to delete user",  
      },  
    );  
  }  
}
```

```
{
  status: 500,
}
);
}
}
```

17. Your Complete CRUD Structure

At this point:

my-next-app/

```
|
|├─ app/
| |
| |├─ api/
| | |├─ users/
| | | |├─ route.js
| | | |├─ [id]/
| | | |├─ route.js
| | |
| |├─ page.js
| |
| |├─ lib/
| | |├─ mongodb.js
| | |
| |├─ .env.local
| |
| |├─ package.json
| |
| |├─ ...
```

Your API becomes:

GET

/api/users

POST

/api/users

PUT

/api/users/:id

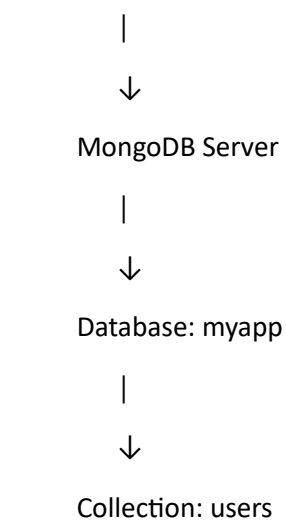
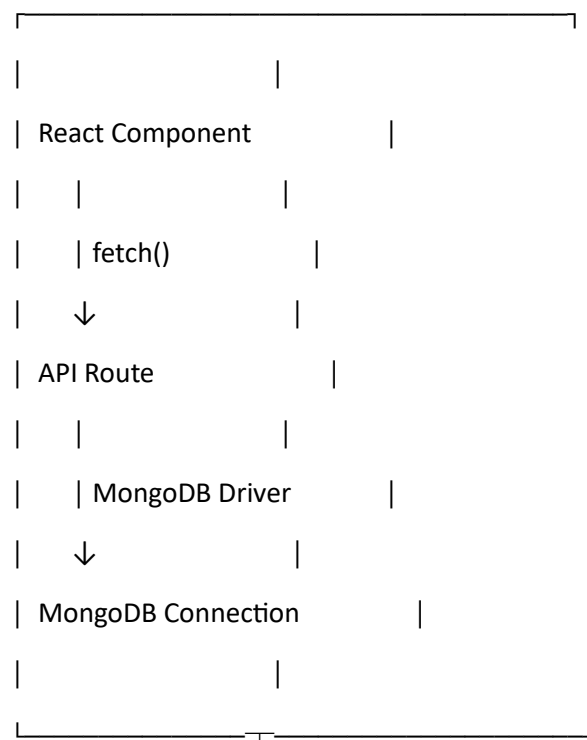
DELETE

/api/users/:id

18. CRUD Flow

The complete architecture is:

NEXT.JS



And CRUD:

CREATE → POST → insertOne()

READ → GET → find()

UPDATE → PUT → updateOne()

DELETE → DELETE → deleteOne()

19. MongoDB Atlas Instead of Local MongoDB

If you're using **MongoDB Atlas**, your .env.local would look like:

MONGODB_URI=mongodb+srv://USERNAME:PASSWORD@cluster.mongodb.net/

MONGODB_DB=myapp

You don't need to change your mongodb.js code.

Only the connection string changes.

20. Important Security Rule

Never do this in a client component:

```
const client = new MongoClient(process.env.MONGODB_URI);
```

And don't expose:

NEXT_PUBLIC_MONGODB_URI=...

The MongoDB connection should remain **server-side**.

Correct:

Client



API Route



MongoDB

Not:

Client



MongoDB directly

The simplest mental model

Remember these **4 layers**:

1. Frontend



2. API Route



3. MongoDB Driver



4. MongoDB

And these **4 CRUD methods**:

POST → insertOne()

GET → find()

PUT → updateOne()

DELETE → deleteOne()